CPEN 208: Introduction to Software Engineering

Combined Project 1 and Project 2 Report

University of Ghana — School of Engineering Sciences

1. Introduction

This project combines the database/application requirements of Project 1 with the API/web-service requirements of Project 2. The system provides student personal information, fee payment records, course enrollment, lecturer-to-course assignments, lecturer-to-TA assignments, authentication, and a dashboard.

2. Technologies Used

PostgreSQL is used for the relational database. Next.js 14 with the App Router, React 18 and TypeScript is used for the web application and API/web service. PostgreSQL is accessed through the pg package. bcryptjs is used to hash passwords and jose is used for signed login sessions.

3. Student Data

The supplied spreadsheet contains 70 student records with Student ID and Name. All 70 records are loaded into the students table. Because the spreadsheet does not contain email addresses, student emails are generated from the Student ID using the format student_id@st.ug.edu.gh. This is a stated project assumption.

4. Database Design

Table	Purpose
students	Stores student personal information.
lecturers	Stores lecturer information.
teaching_assistants	Identifies students serving as teaching assistants.
courses	Stores course information.
course_enrollments	Associates students with courses.
course_lecturers	Associates lecturers with courses.
course_tas	Associates teaching assistants with courses.
fee_invoices	Stores the amount each student is required to pay.
fee_payments	Stores payments made against invoices.
app_users	Stores web-app account information and password hashes.

5. Outstanding Fees Function

The PostgreSQL function get_outstanding_fees() calculates total amount due, total amount paid and the remaining outstanding amount for every student. It returns the result as a JSONB array. The /api/fees/outstanding endpoint calls this function directly.

6. Web Application

The Next.js application provides register and login pages and a protected dashboard. The dashboard displays student information, courses and outstanding fees. Authentication uses an HTTP-only signed session cookie. Passwords are stored as bcrypt hashes rather than plain text.

7. API/Web Service

Endpoint	Function
GET /api/students	Retrieve student personal information.
POST /api/students	Create a student record.
GET /api/courses	Retrieve courses.
GET /api/enrollments	Retrieve course enrollments.
POST /api/enrollments	Enroll a student in a course.
GET /api/fees	Retrieve invoices, payments and outstanding balances.
POST /api/fees	Record a fee payment.
GET /api/fees/outstanding	Return the PostgreSQL outstanding-fees JSON array.
GET /api/assignments	Retrieve lecturer and TA course assignments.
POST /api/assignments	Assign a lecturer or TA to a course.

8. Assumptions

The spreadsheet supplies only student IDs and names, so student emails are generated. Four sample lecturers, six sample courses and teaching-assistant assignments are created to demonstrate the required relationships. The first twelve students are used as sample teaching assistants. A sample first-semester invoice is created for each student so that the outstanding-fees function can be demonstrated.

9. Visual Design

The interface uses warm brown, terracotta, cream and soft-orange colors. A natural serif font stack is used for readability and an academic visual feel. The layout is responsive and uses simple cards, tables and navigation.

10. Running the System

Create the PostgreSQL database, run db/01_schema.sql and db/02_seed.sql, configure DATABASE_URL and JWT_SECRET in .env.local, run npm install, and then run npm run dev. The application is available at http://localhost:3000. Before final submission, create a PostgreSQL backup with pg_dump and add the backup and the final GitHub URL to the submission repository.

11. Conclusion

The implementation provides the relational database, sample class data, outstanding-fees JSON function, authenticated Next.js interface and API/web service requested by the two CPEN 208 projects. The repository also contains the database scripts and instructions required to reproduce the system.